Les tuples

Les types construits

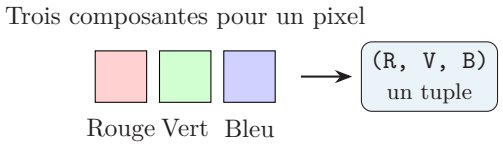
1. Comment une image est-elle constituée ?

Une image numérique est composée d'un très grand nombre de petits carrés appelés pixels. Chaque pixel possède une position et une couleur.

Dans une image couleur classique, la couleur d'un pixel est décrite par trois nombres :

- la quantité de rouge (R) ;
- la quantité de vert (V) ;
- la quantité de bleu (B).

Chaque composante est un entier compris entre 0 et 255.



Exemple

Le tuple `(255, 0, 0)` représente un rouge pur, `(0, 255, 0)` un vert pur et `(0, 0, 0)` le noir.

DÉFINITION

Un tuple est une collection ordonnée de valeurs. En Python, on l'écrit entre parenthèses, avec des virgules.

```
pixel = (125, 200, 80)
rouge = pixel[0]
vert = pixel[1]
bleu = pixel[2]
```

Les indices commencent à 0. Un tuple n'est pas modifiable directement : on peut lire ses éléments, mais pour obtenir une nouvelle couleur, on construit un nouveau tuple.

Exercice 1

1) Dans le tuple `pixel = (125, 200, 80)`, quelle valeur correspond au vert ?
2) Écrire une instruction qui affiche la composante bleue du tuple `pixel`.
3) Donner le tuple correspondant au blanc, puis celui correspondant au noir.
4) Expliquer pourquoi `pixel[0] = 0` provoque une erreur.

2. Charger et observer une image

Dans une première cellule du notebook, exécuter le code suivant.

```
from PIL import Image
import requests
from io import BytesIO
import basthon

URL_IMAGE = ("https://raw.githubusercontent.com/Olivier43120/basthon_images/main/la-tour-poudriere
      -de-Prague.jpg")
```

```
reponse = requests.get(URL_IMAGE)
reponse.raise_for_status()

image = Image.open(BytesIO(reponse.content)).convert("RGB")

print("Dimensions :", image.size)
print("Type de l'image :", image.mode)
image.show()
```

> **Attention**
>
> Si `requests.get` renvoie une erreur, vérifiez la connexion et l'adresse. Le serveur qui héberge l'image doit autoriser les requêtes provenant de Basthon. Une autre solution consiste à déposer l'image dans le notebook avec le bouton d'ouverture des fichiers, puis à utiliser `Image.open("EP24-08.2.png")`.

Exercice 2

1) Que renvoie `image.size` ? À quoi correspondent les deux nombres ?
2) Que renvoie `image.mode` ?
3) À l'aide de `image.getpixel((0, 0))`, afficher le tuple du pixel situé en haut à gauche.
4) Afficher le tuple du pixel situé au centre de l'image. On pourra utiliser :

```
largeur, hauteur = image.size
print(image.getpixel((largeur // 2, hauteur // 2)))
```

5) Combien de composantes contient le tuple renvoyé par `getpixel` ?

3. Éclaircir et assombrir une image

Pour éclaircir une image, on peut augmenter chacune des trois composantes de chaque pixel. Il faut toutefois éviter de dépasser 255. Pour assombrir, on diminue les composantes sans descendre sous 0.

A. Comprendre le traitement d'un pixel

On souhaite ajouter 40 à chaque composante du pixel `(220, 100, 30)`. La nouvelle couleur ne peut pas être `(260, 140, 70)`, car 260 dépasse la valeur maximale autorisée.
On utilise donc `min(255, valeur + 40)`.

Exercice 3

1) Calculer à la main le nouveau tuple obtenu à partir de `(220, 100, 30)` en ajoutant 40 à chaque composante, sans dépasser 255.
2) Compléter la fonction ci-dessous. Elle reçoit une image et une valeur entière, puis renvoie une nouvelle image éclaircie.

```
def eclaircir(image, valeur):
    largeur, hauteur = image.size
    resultat = Image.new("RGB", image.size)

    for x in range(largeur):
        for y in range(hauteur):
```

```
            r, v, b = image.getpixel((x, y))

            # A COMPLETER : calculer les trois composantes
            nouveau_pixel = (
                ...,
                ...,
                ...
            )

            resultat.putpixel((x, y), nouveau_pixel)

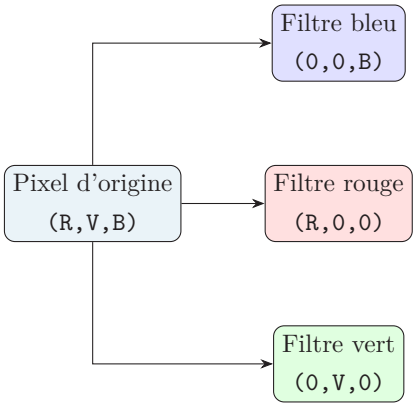
    return resultat
```

Exercice 4

1) Tester `eclaircir(image, 40)` et afficher le résultat.
2) Écrire une fonction `assombrir(image, valeur)` sur le même modèle. On utilisera `max(0, composante - valeur)`.
3) Tester plusieurs valeurs et décrire l'effet observé.
4) Pourquoi faut-il créer une nouvelle image plutôt que modifier directement le tuple du pixel ?

4. Créer des filtres de couleur

On peut fabriquer des effets simples en conservant une seule composante de chaque pixel.



Exercice 5

1) Compléter une fonction `filtre_rouge(image)` qui remplace chaque pixel `(r,v,b)` par `(r,0,0)`.
2) Adapter cette fonction pour créer un filtre vert, puis un filtre bleu.
3) Comparer les trois images obtenues. Quelle information de l'image d'origine est conservée par chaque filtre ?

5. Transformer une image en niveaux de gris

Une méthode simple consiste à calculer la moyenne des trois composantes :

$$g = \left\lfloor \frac{R+V+B}{3} \right\rfloor .$$

On remplace alors le pixel `(R,V,B)` par le tuple `(g,g,g)`.

Exercice 6

1) Calculer la valeur de gris correspondant au pixel `(30, 120, 210)`.
2) Compléter la fonction suivante.

```
def niveaux_de_gris(image):
    largeur, hauteur = image.size
    resultat = Image.new("RGB", image.size)

    for x in range(largeur):
        for y in range(hauteur):
            r, v, b = image.getpixel((x, y))

            gris = ... # A COMPLETER

            resultat.putpixel((x, y), (gris, gris, gris))

    return resultat
```

Exercice 7

Afficher l'image en niveaux de gris et expliquer pourquoi les trois composantes d'un pixel sont alors égales.

6. Stéganographie : cacher un message dans une image

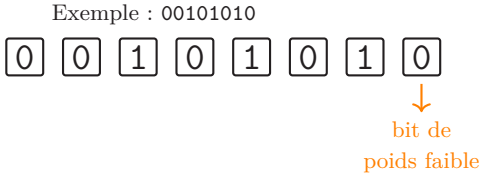
La stéganographie consiste à dissimuler une information dans un support qui semble ordinaire. Dans cette activité, le support est une image et le message est encodé dans les bits de poids faible des composantes RVB.

A. Écriture binaire et bit de poids faible

Un entier entre 0 et 255 peut être écrit sur 8 bits. Par exemple :

$$42 = 00101010_2.$$

Le bit le plus à droite est appelé bit de poids faible. Le remplacer par 0 ou 1 modifie la valeur de la composante de 0 ou de 1 seulement.



En Python, les opérateurs binaires permettent d'agir sur les bits :

- `valeur & 254` met le dernier bit à 0 ;
- `(valeur & 254) | bit` impose ensuite ce dernier bit à `bit`.

Par exemple, si `valeur = 42`, alors remplacer son bit de poids faible par 1 donne 43. La modification est très petite et généralement difficile à remarquer à l'œil nu.

B. Préparer le message

Pour stocker du texte, on le transforme d'abord en octets avec l'encodage UTF-8. Chaque octet est ensuite représenté par 8 bits.

```
message = "Bonjour"
octets = message.encode("utf-8")
print(octets)
```

Exercice 8

1) Exécuter le code. Quel est le type de `octets` ?
2) À quoi sert la méthode `encode("utf-8")` ?
3) Combien de bits faut-il pour stocker un message de 7 caractères ASCII ?

C. Cacher le message (aller)

Le code ci-dessous ajoute une information de longueur au début du message. Cela permet de retrouver précisément la fin du texte, même si le message contient des caractères accentués.

```
def cacher_message(image, message):
    resultat = image.copy()

    donnees = message.encode("utf-8")
    longueur = len(donnees)

    # Les 4 premiers octets indiquent la longueur
    entete = longueur.to_bytes(4, "big")
    bits = "".join(format(octet, "08b")
                for octet in entete + donnees)

    largeur, hauteur = resultat.size
    capacite = largeur * hauteur * 3

    if len(bits) > capacite:
        raise ValueError("Image trop petite pour ce message")

    indice = 0

    for y in range(hauteur):
        for x in range(largeur):
            pixel = list(resultat.getpixel((x, y)))

            for canal in range(3):
                if indice < len(bits):
                    pixel[canal] = (
                        (pixel[canal] & 254)  int(bits[indice]))indice += 1resultat.putpixel((x, y),
                             tuple(pixel))if indice == len(bits):return resultatreturn resultat
```

Exercice 9

1) Pourquoi utilise-t-on `image.copy()` ?
2) Pourquoi la capacité maximale est-elle de trois bits par pixel ?
3) À quoi sert le tuple `(R,V,B)` dans cette fonction ?
4) Exécuter :

```
message_secret = "Bonjour !"
image_secrete = cacher_message(image, message_secret)
image_secrete.show()
```

5) L'image semble-t-elle différente de l'originale ?
6) Enregistrer l'image secrète au format PNG.

D. Retrouver le message (retour)

Pour lire le message, on récupère les bits de poids faible dans le même ordre. Les 32 premiers bits donnent la longueur du message ; les bits suivants constituent les octets du texte.

```
def lire_message(image):
    largeur, hauteur = image.size
    bits = ""

    for y in range(hauteur):
        for x in range(largeur):
            pixel = image.getpixel((x, y))

            for canal in range(3):
                bits += str(pixel[canal] & 1)

    # Lire les 32 bits de l'entete
    longueur = int(bits[:32], 2)

    # Lire exactement longueur octets
    nombre_bits = longueur * 8
    bits_message = bits[32:32 + nombre_bits]

    donnees = bytearray()

    for i in range(0, len(bits_message), 8):
        octet = bits_message[i:i+8]
        donnees.append(int(octet, 2))

    return donnees.decode("utf-8")
```

Exercice 10

1) Exécuter `lire_message(image_secrete)`.
2) Vérifier que le texte retrouvé est identique au message initial.
3) Tester avec un message contenant des accents.
4) Que se passe-t-il si l'on essaie de lire le message dans l'image originale, qui ne contient pas de message caché ?
5) Pourquoi faut-il conserver l'image au format PNG et éviter de la réenregistrer en JPEG ?

Attention

Cette méthode est une démonstration pédagogique, pas un moyen de sécuriser un secret. Elle ne chiffre pas le message : une personne qui connaît la méthode peut le retrouver. Elle suppose aussi que l'image n'a pas été redimensionnée ni recompressée avec perte. Le format PNG convient à cette expérience.

7. Enregistrer et télécharger les images

Une image créée peut être enregistrée dans le système de fichiers du notebook, puis téléchargée sur l'ordinateur.

```
image_claire = eclaircir(image, 40)
image_grise = niveaux_de_gris(image)
image_secrete = cacher_message(image, "Bonjour !")

image_claire.save("image_claire.png")
image_grise.save("image_grise.png")
image_secrete.save("image_secrete.png")

basthon.download("image_claire.png")
basthon.download("image_grise.png")
basthon.download("image_secrete.png")
```

Exercice 11

Télécharger les trois images. Les ouvrir depuis votre ordinateur et vérifier qu'elles ont bien été enregistrées.

8. Défi : à vous de créer un filtre !

Choisir une transformation supplémentaire et la programmer :

- inverser les couleurs : `(255-R, 255-V, 255-B)` ;
- ne conserver que les pixels dont la composante rouge dépasse un seuil ;
- échanger les composantes rouge et bleue : `(B,V,R)` ;
- créer un effet « négatif » ;
- proposer une transformation personnelle.

Le programme devra recevoir une image et renvoyer une nouvelle image. Il devra utiliser des tuples pour représenter les couleurs.

9. Bilan

Répondre avec des phrases courtes et précises.

1) Qu'est-ce qu'un tuple en Python ?
2) Comment accéder à la composante verte d'un pixel nommé `pixel` ?
3) Pourquoi une couleur RVB est-elle représentée par trois valeurs ?
4) Pourquoi utilise-t-on deux boucles pour parcourir tous les pixels d'une image ?
5) Pourquoi crée-t-on une nouvelle image lors d'un filtre ?
6) Quel est le principe du bit de poids faible dans la stéganographie ?
7) Quelle différence faites-vous entre cacher un message et chiffrer un message ?